Algorithmique 1
Les types de données abstraits

Olivier Markowitch

Les structures de données

Les algorithmes ont besoin de structures de données appropriées pour stocker les données qu'ils manipulent

Une structure de données est une organisation de l'information que l'on tente de rendre optimale (en taille des données stockées ou en temps de traitement des données) par rapport à l'usage qui en sera fait

Les structures de données

Les traitements de base sont l'ajout d'une donnée, la suppression d'une donnée et éventuellement la modification d'une donnée

Exemples de structures de données :

- le vecteur ou tableau à une dimension (dont les éléments sont souvent indicés de 0 à $n-1$)
- la matrice, tableau à deux dimensions
- les tableaux à n dimensions

Les listes

Autre exemple de structure de données : **la liste**

⚠ La liste en algorithmique n'est pas la liste python

Une liste est un ensemble dynamique d'objets ordonnés accessibles relativement les uns aux autres, sur base de leur position au sein de la liste

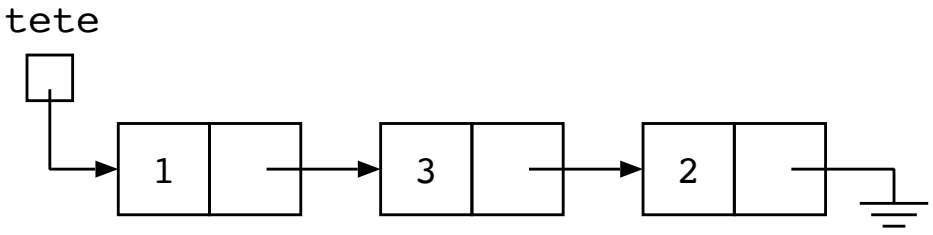
Les listes sont utiles quand on doit pouvoir ajouter, supprimer ou insérer rapidement (en temps constant) une information au sein d'un ensemble d'information

Les listes

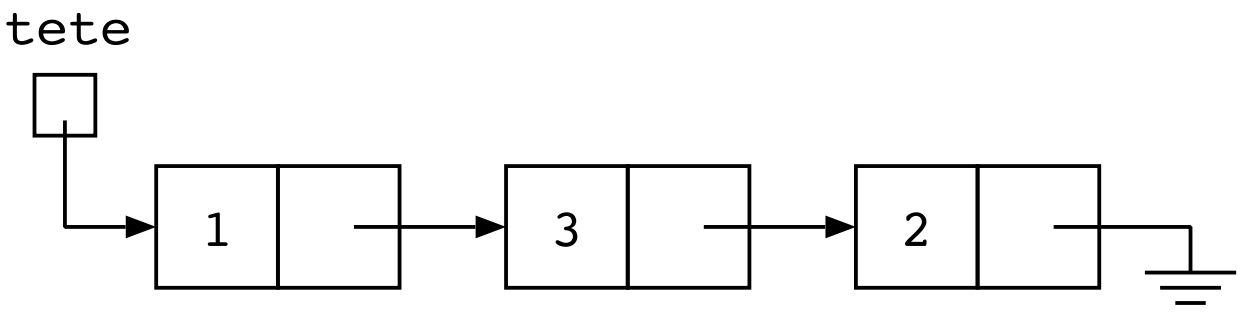
Une liste simple est composée d'une tête de liste (qui donne accès à la liste) et d'une chaine d'éléments qui composent la liste

Chaque élément est composé d'une information et d'une référence vers un autre élément, appelé élément suivant

Le dernier élément mentionne comme « suivant » une référence particulière qui indique que la liste est terminée



Liste chaînée



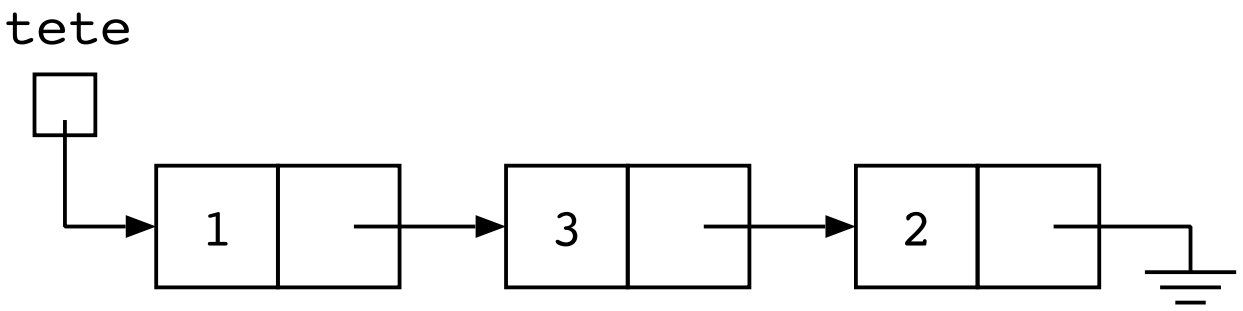
En pratique une telle liste chaînée peut être réalisée de différentes manières

Liste chaînée

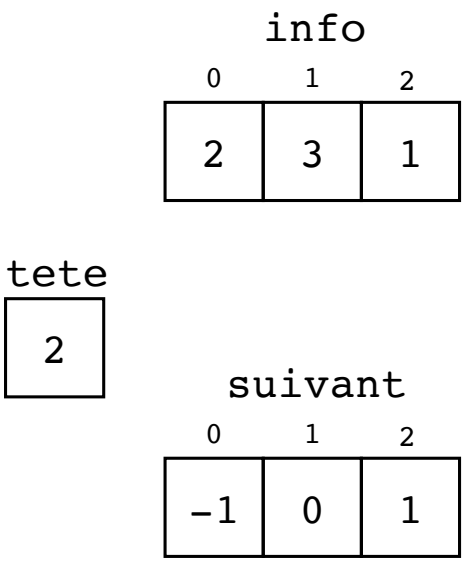
Exemples :

- au moyen de pointeurs (sur base d'allocation de la mémoire)
- au moyen de tableaux
- au moyen de nœuds qui font référence à des nœuds

Liste chaînée au moyen de pointeurs



Liste chaînée au moyen de tableaux



info

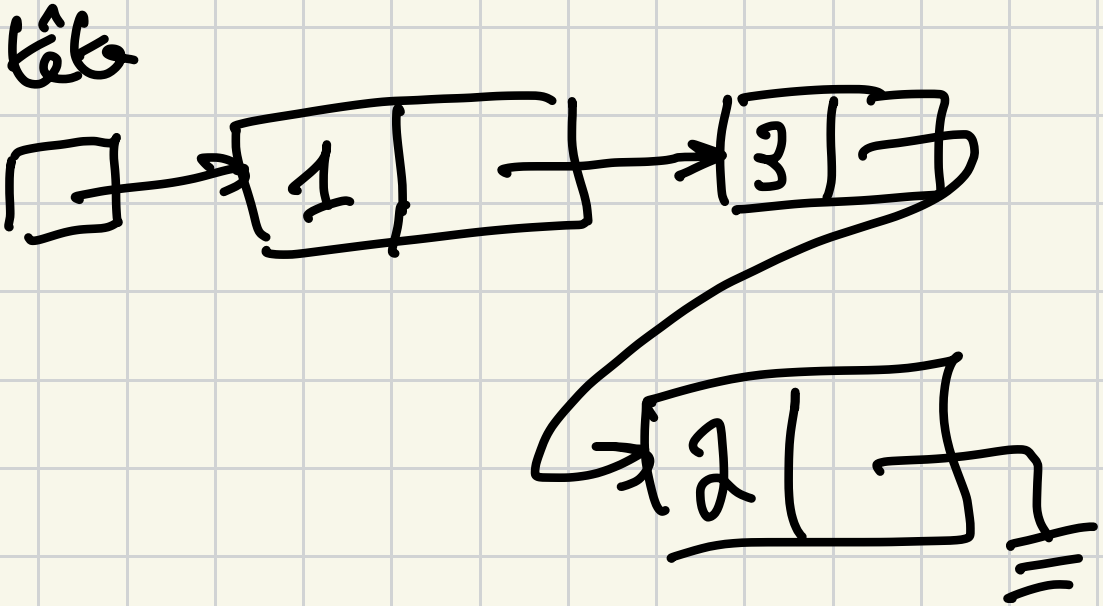
0	1	2
2	3	1

tete

2

suivant

0	1	2
-1	0	1

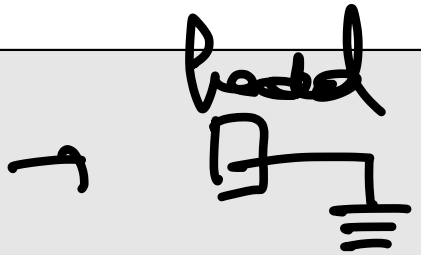


Liste chaînée au moyen de référence à des nœuds

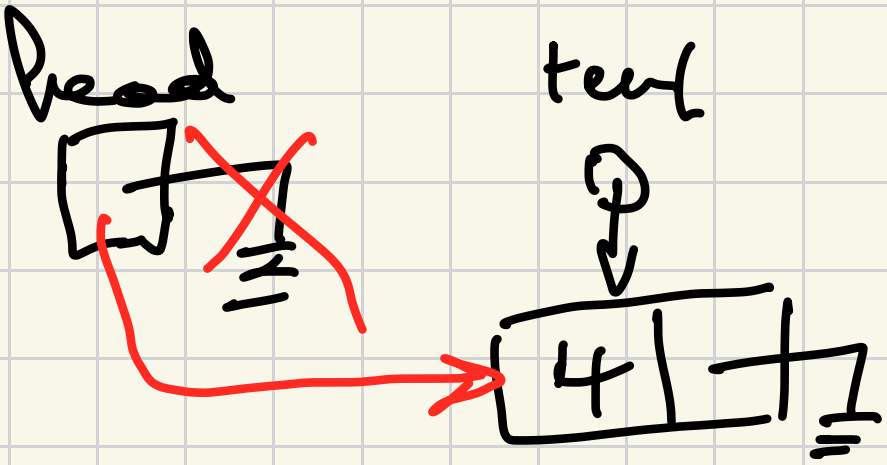
```
class Node:
    def __init__(self,initdata):
        self.data = initdata
        self.next = None

    def getData(self):
        return self.data

    def getNext(self):
        return self.next

    def setData(self,newdata):
        self.data = newdata

    def setNext(self,newnext):
        self.next = newnext
```

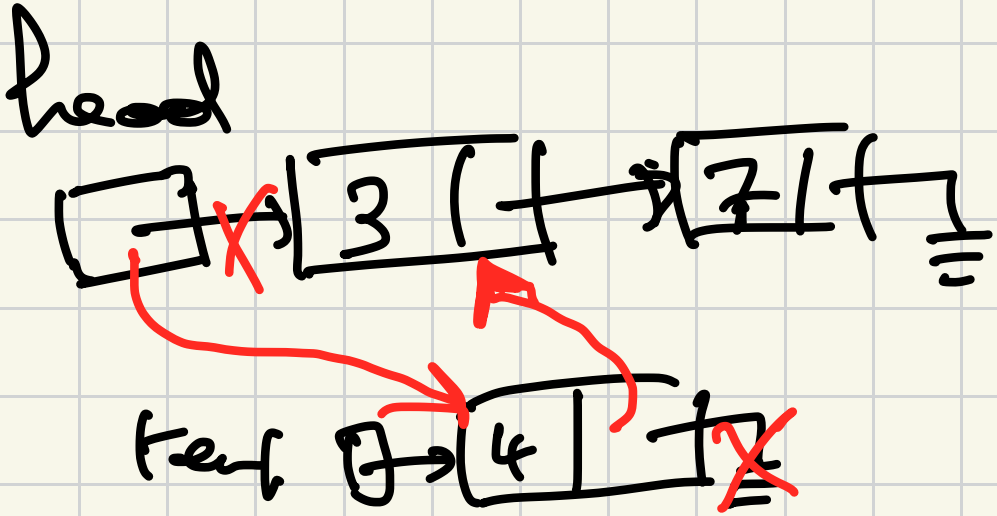
Liste chaînée au moyen de référence à des nœuds

```
class UnorderedList:
    def __init__(self):
        self.head = None

    def isEmpty(self):
        return self.head == None

    def add(self,item):
        temp = Node(item)
        temp.setNext(self.head)
        self.head = temp

    def addAfter(self,base,item):
        temp = Node(item)
        temp.setNext(base.getNext())
        base.setNext(temp)
```

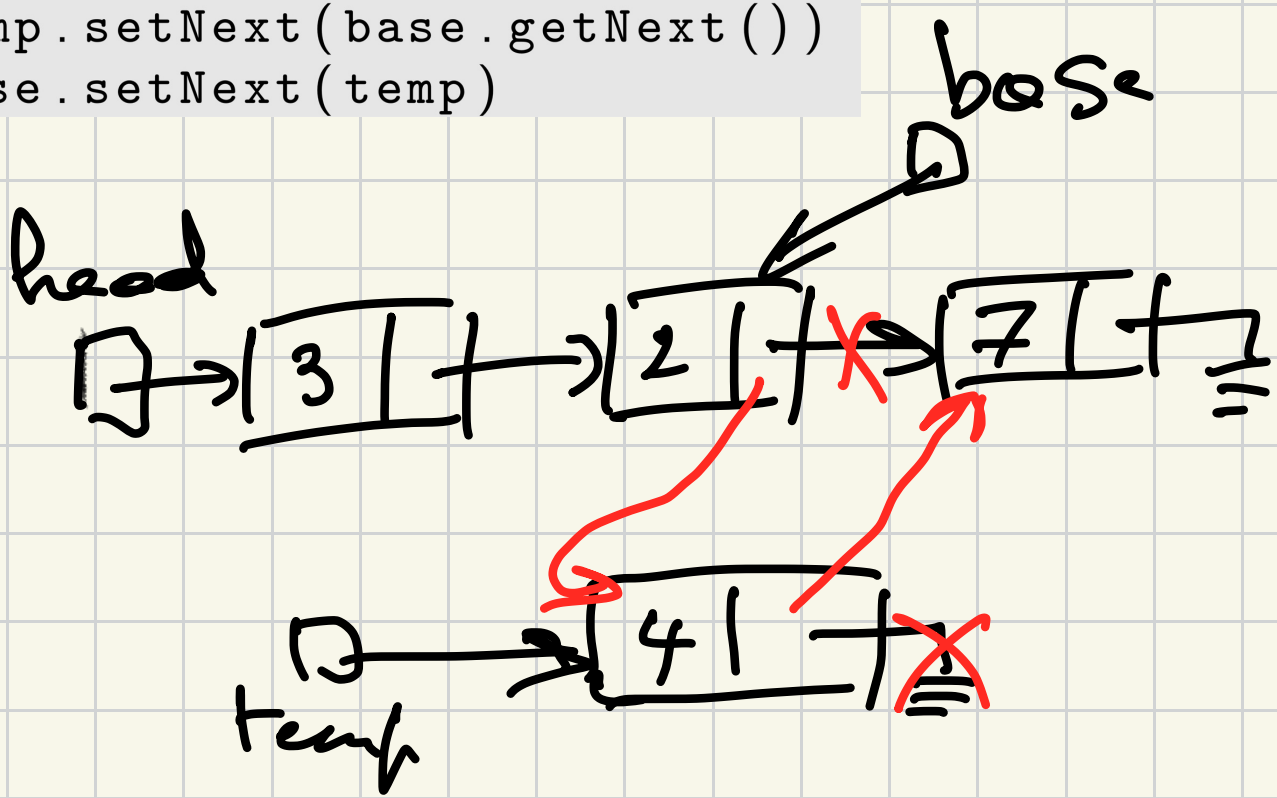


```python
def add(self,item):
    temp = Node(item)
    temp.setNext(self.head)
    self.head = temp
```





```python
def addAfter(self,base,item):
    temp = Node(item)
    temp.setNext(base.getNext())
    base.setNext(temp)
```



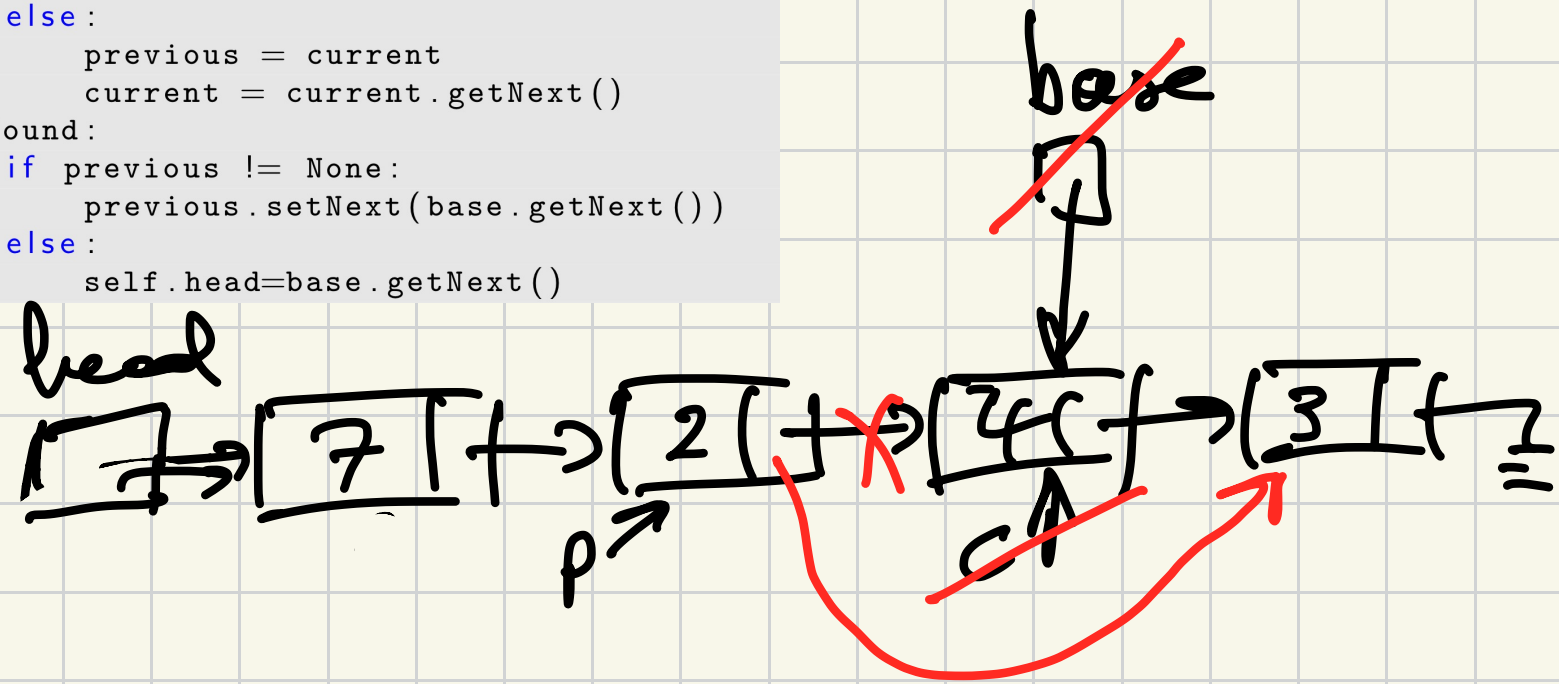
Liste chaînée au moyen de référence à des nœuds

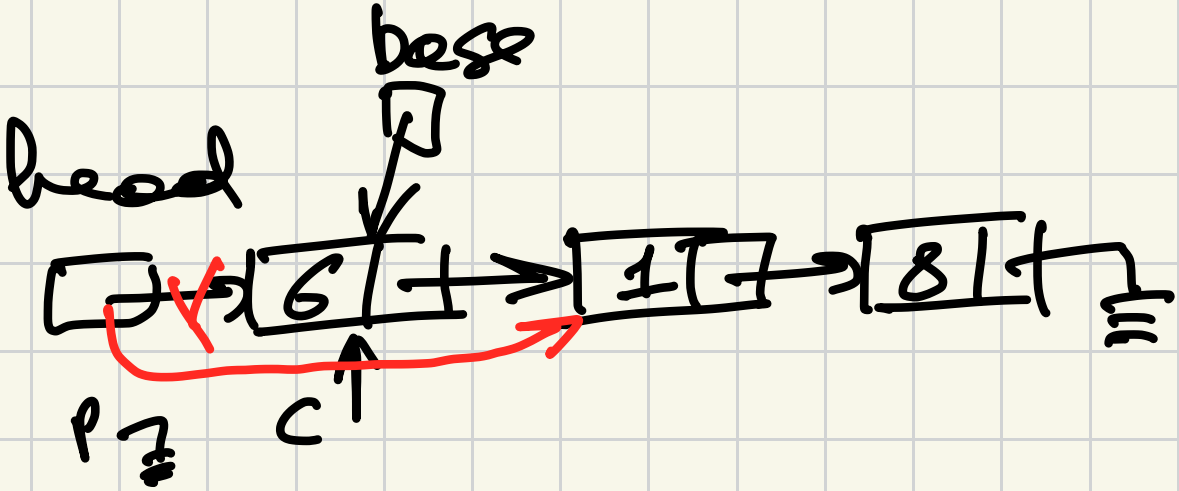
```
    def length(self):
        current = self.head
        count = 0
        while current != None:
            count = count + 1
            current = current.getNext()
        return count

    def search(self,item):
        current = self.head
        found = False
        while current != None and not found:
            if current.getData() == item:
                found = True
            else:
                current = current.getNext()
        return found
```

Liste chaînée au moyen de référence à des nœuds

```
    def remove(self,base):
        previous = None
        current = self.head
        found = False
        while current != None and not found:
            if current is base:
                found = True
            else:
                previous = current
                current = current.getNext()
        if found:
            if previous != None:
                previous.setNext(base.getNext())
            else:
                self.head=base.getNext()
```

```python
def remove(self,base):
    previous = None
    current = self.head
    found = False
    while current != None and not found:
        if current is base:
            found = True
        else:
            previous = current
            current = current.getNext()
    if found:
        if previous != None:
            previous.setNext(base.getNext())
        else:
            self.head=base.getNext()
```





Type de donnée abstrait

Ces différentes implémentations de listes illustrent bien que la notion de liste est indépendante de l'implémentation sous-jacente

Nous allons manipuler une série d'outils algorithmiques (tels que la liste, etc.)

Le concept, ce à *quoi* l'outil sert, sera présenté distinctement des détails d'implémentation, *comment* l'outil est réalisé

Type de donnée abstrait

Ainsi un tel outil pourra être utilisé indifféremment de la façon dont il est conçu

Si nous devons concevoir, par exemple, un algorithme qui doit manipuler une liste, notre algorithme pourra donc fonctionner indépendamment de la façon dont cette liste est réalisée

Un type de données défini suivant une telle approche est appelé un *type de données abstrait* (ou *abstract data type* en anglais)

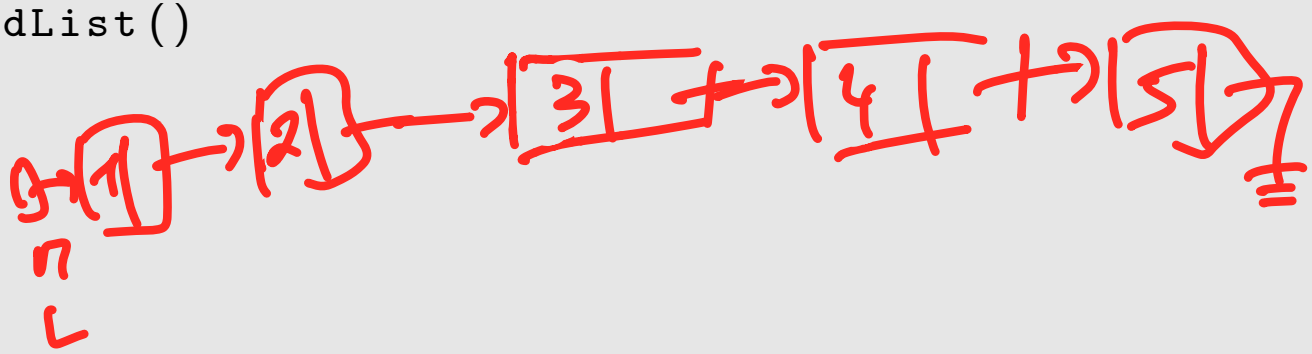
Passage de paramètre ou assignation d'une liste

Considérons une classe représentant une liste linéaire simple, que nous instantions sous forme d'un objet L de type Liste et envisageons les deux situations suivantes :

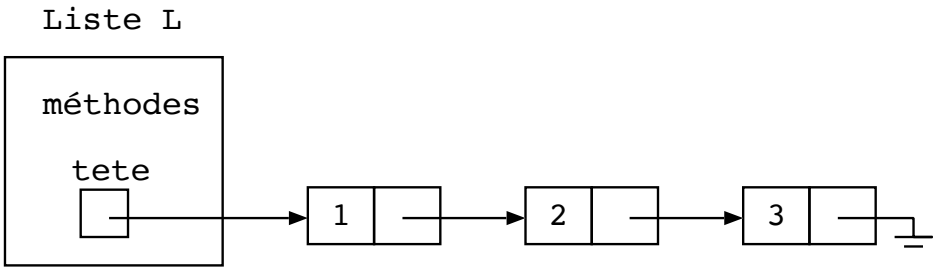
- la liste est passée comme paramètre d'une fonction
- assignation de la liste

Passage de paramètre ou assignation d'une liste

```
>>> L=UnorderedList()
>>> L.add(5)
>>> L.add(4)
>>> L.add(3)
>>> L.length()
3
>>> M=L
>>> M.add(2)
>>> L.length()
4
>>> M is L
True
>>> def ajout(liste):
    liste.add(1)

>>> ajout(M)
>>> M.length()
5
>>> L.search(1)
True
```



Copie de listes : L1=L

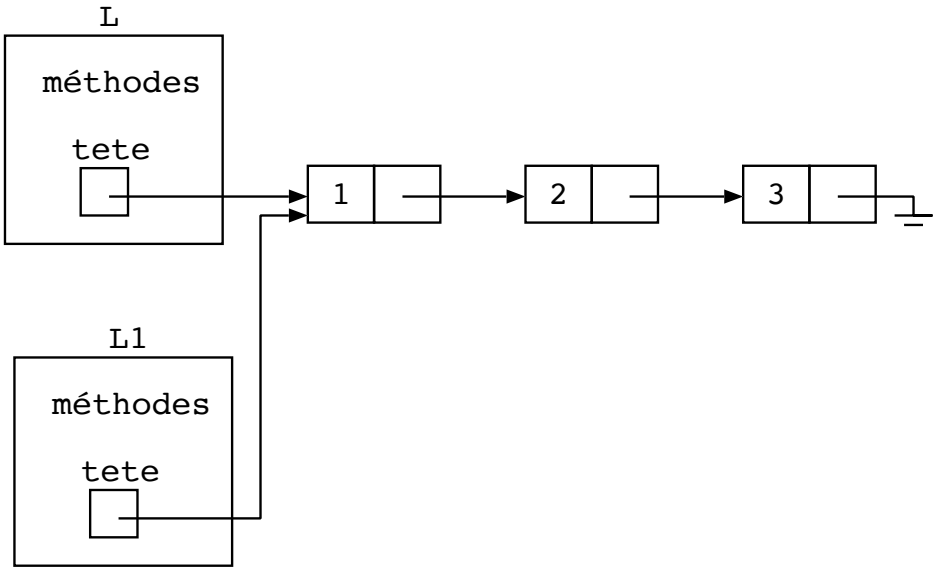


Passage de paramètre ou assignation d'une liste

```
>>> import copy
>>> N=copy.copy(M)
>>> O=copy.deepcopy(M)
>>> N is M
False
>>> O is M
False
>>> noeud=N.head
>>> noeud=noeud.getNext()
>>> noeud.getData()
2
>>> L.addAfter(noeud,7)
>>> L.length()
6
>>> M.length()
6
>>> N.length()
6
>>> O.length()
5
```

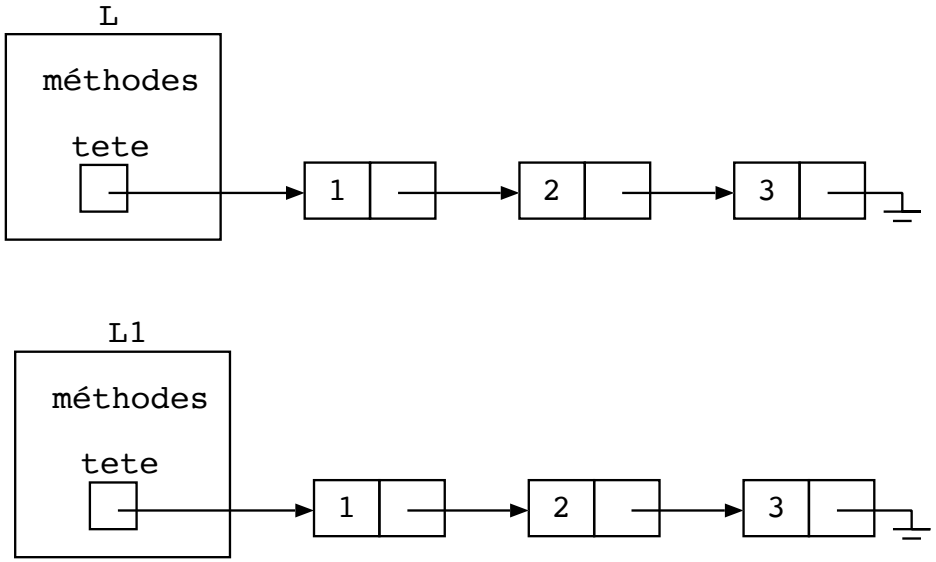
Copie de listes : copy

L1=copy.copy(L)



Copie de listes : deepcopy

L1=copy.deepcopy(L)

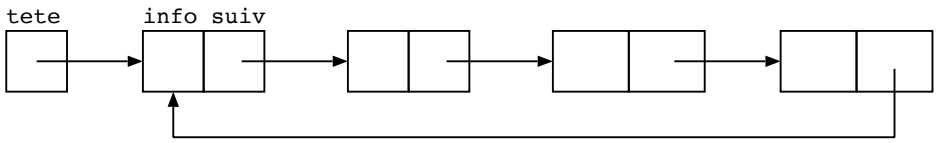


Variantes sur les listes

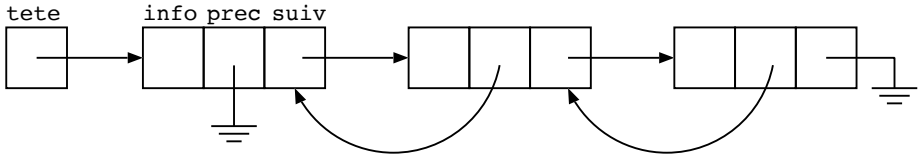
En plus des listes chaînées linéaire, il existe des variantes qui peuvent être utiles en fonction des besoins :

- liste circulaire (avec élément bidon)
- liste bidirectionnelle
- liste circulaire et bidirectionnelle (cf séances d'exercices)

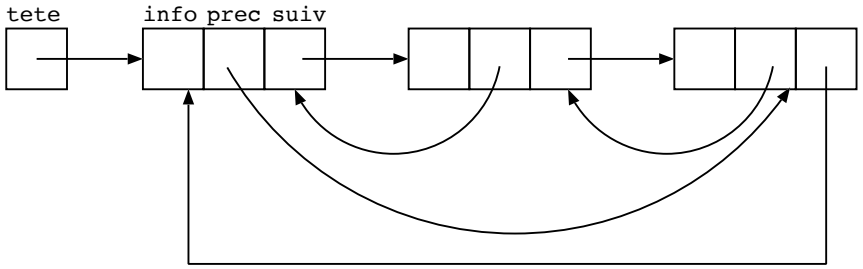
Variantes sur les listes



Liste circulaire



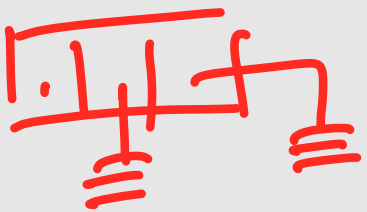
Liste bidirectionnelle



Liste circulaire bidirectionnelle

Liste bidirectionnelle : Node

```
class Node:
    def __init__(self,initdata):
        self.data = initdata
        self.next = None
        self.previous = None

    def getData(self):
        return self.data

    def getNext(self):
        return self.next

    def getPrevious(self):
        return self.previous
```



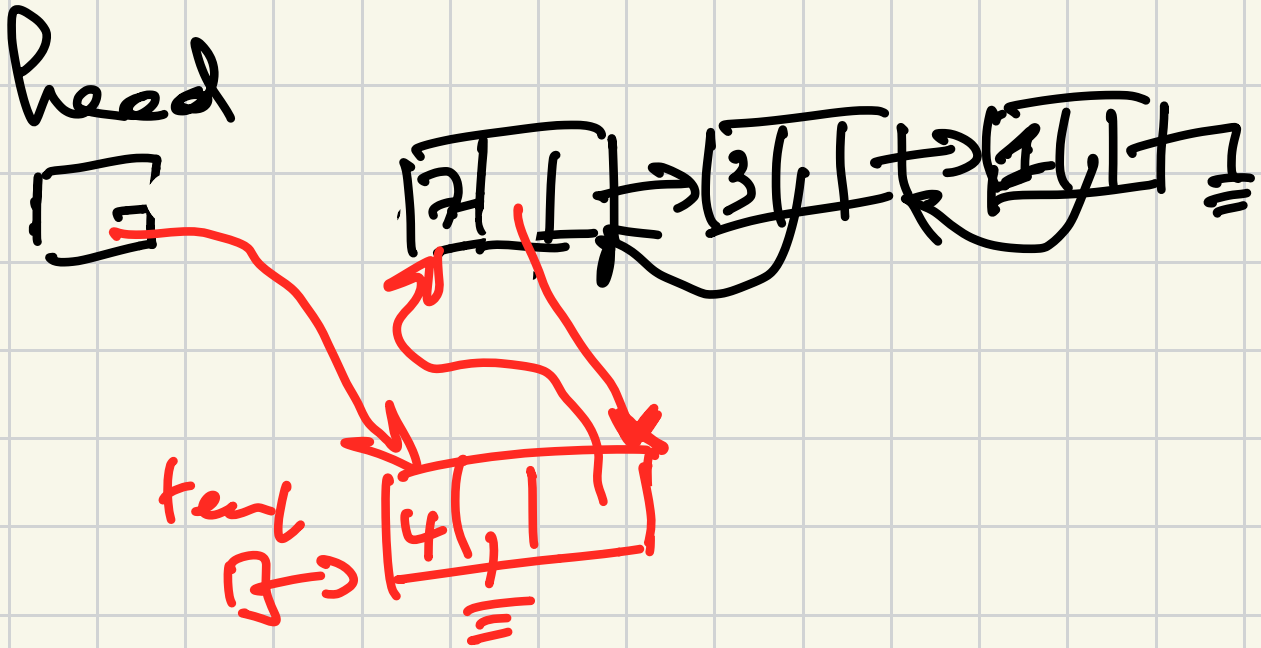
Liste bidirectionnelle : Node

```
    def setData(self,newdata):
        self.data = newdata

    def setNext(self,newnext):
        self.next = newnext

    def setPrevious(self,newprevious):
        self.previous = newprevious
```

Liste bidirectionnelle

```
class UnorderedList:
    def __init__(self):
        self.head = None
        self.count=0

    def isEmpty(self):
        return self.head == None

    def add(self,item):
        temp = Node(item)
        temp.setNext(self.head)
        if self.head != None:
            self.head.setPrevious(temp)
        self.head = temp
        self.count = self.count + 1
```

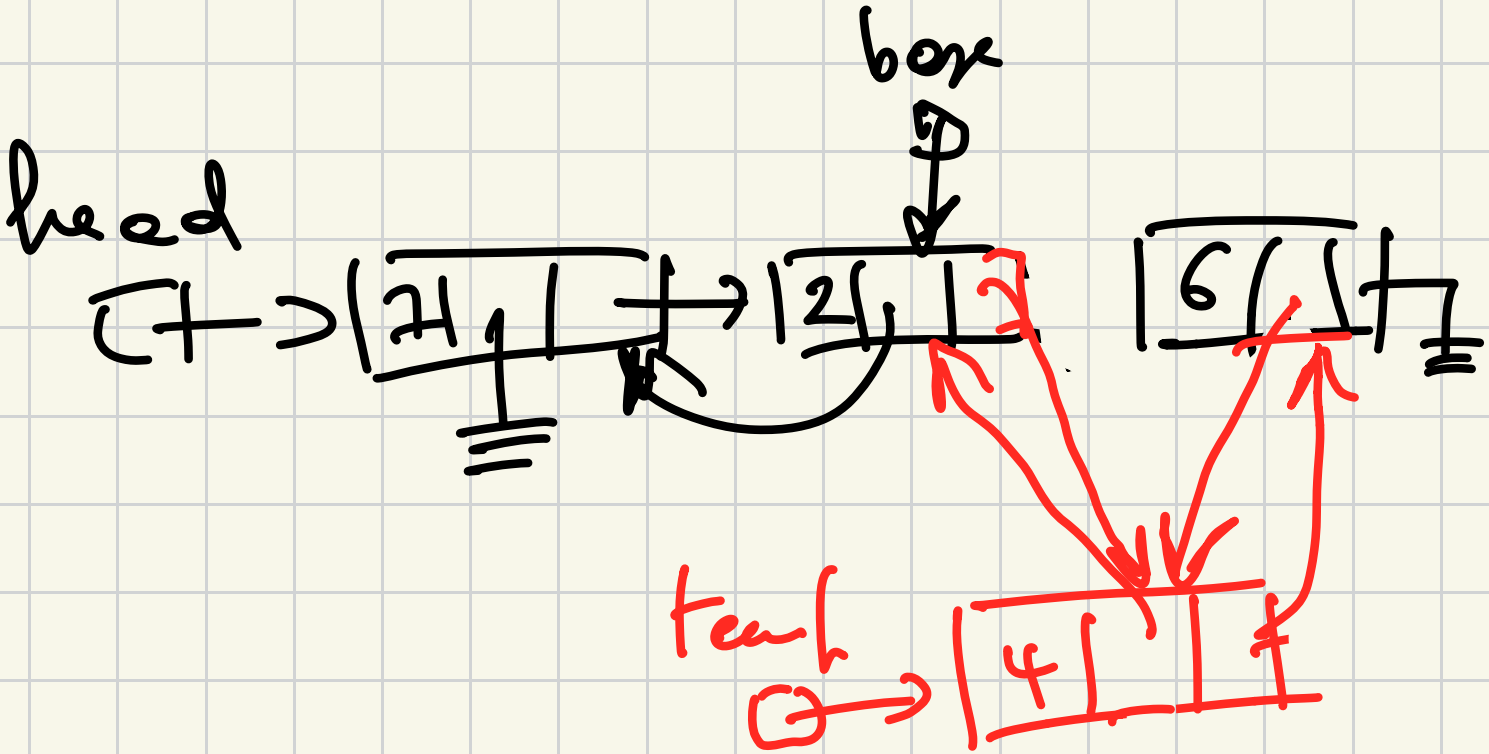
```python
def add(self,item):
    temp = Node(item)
    temp.setNext(self.head)
    if self.head != None:
        self.head.setPrevious(temp)
    self.head = temp
    self.count = self.count + 1
```



Liste bidirectionnelle

```
    def addAfter(self,base,item):
        temp = Node(item)
        temp.setPrevious(base)
        temp.setNext(base.getNext())
        if base.getNext() != None:
            base.getNext().setPrevious(temp)
        base.setNext(temp)
        self.count = self.count + 1

    def length(self):
        return self.count

    def search(self,item):
        current = self.head
        found = False
        while current != None and not found:
            if current.getData() == item:
                found = True
            else:
                current = current.getNext()
        return found
```

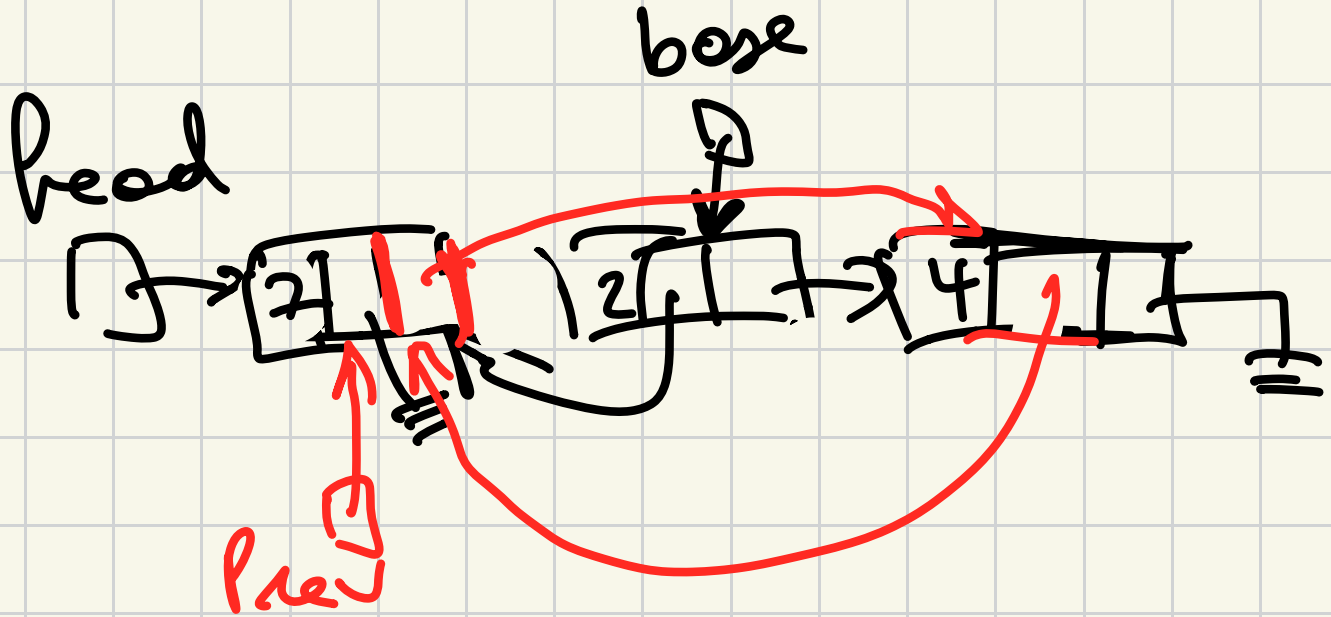
```python
def addAfter(self,base,item):
    temp = Node(item)
    temp.setPrevious(base)
    temp.setNext(base.getNext())
    if base.getNext() != None:
        base.getNext().setPrevious(temp)
    base.setNext(temp)
    self.count = self.count + 1
```



Liste bidirectionnelle

```
    def remove(self,base):
        previous = base.getPrevious()
        if base.getNext() != None:
                base.getNext().setPrevious(previous)
        if previous != None:
            previous.setNext(base.getNext())
        else:
            self.head=base.getNext()
        self.count = self.count - 1
```

```python
def remove(self,base):
    previous = base.getPrevious()
    if base.getNext() != None:
            base.getNext().setPrevious(previous)
    if previous != None:
        previous.setNext(base.getNext())
    else:
        self.head=base.getNext()
    self.count = self.count - 1
```

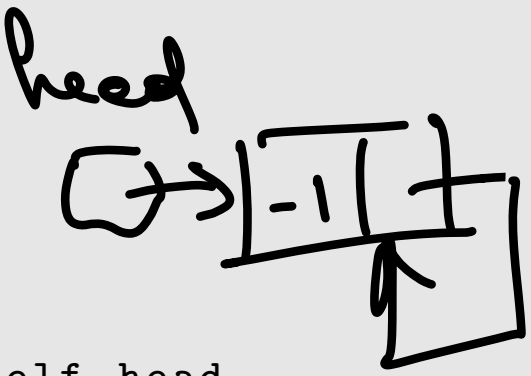


Liste circulaire : Node

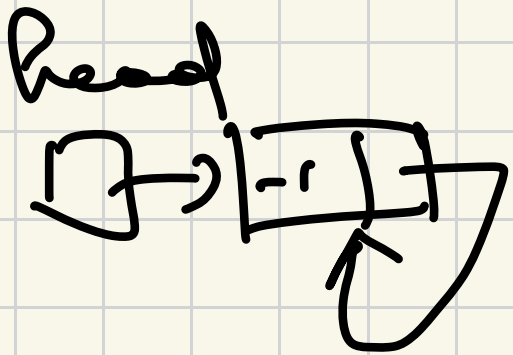
```
class Node:
    def __init__(self,initdata):
        self.data = initdata
        self.next = None

    def getData(self):
        return self.data

    def getNext(self):
        return self.next

    def setData(self,newdata):
        self.data = newdata

    def setNext(self,newnext):
        self.next = newnext
```

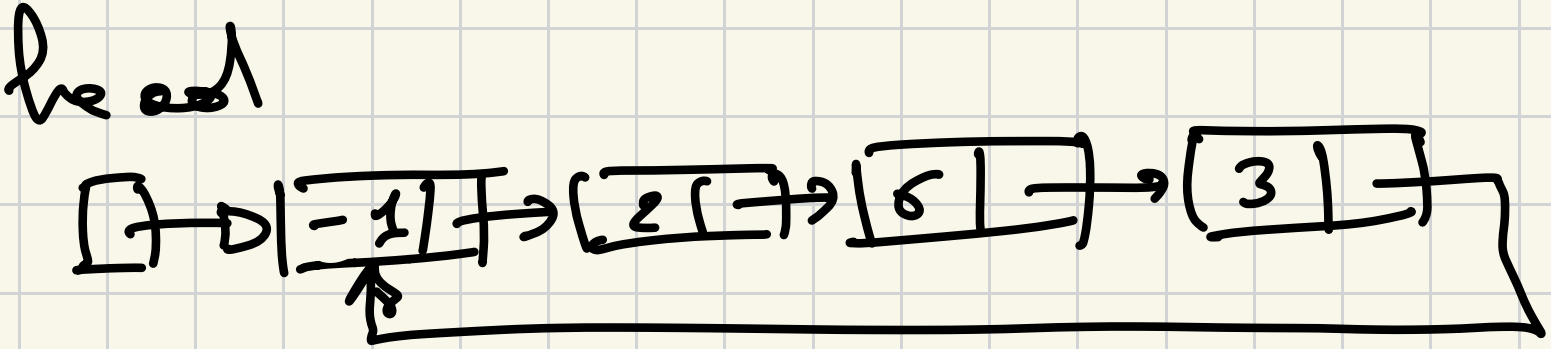
Liste circulaire

```
class UnorderedList:
    def __init__(self):
        self.head = Node(-1)
        self.head.setNext(self.head)
        self.count = 0

    def isEmpty(self):
        return self.head.getNext() == self.head

    def add(self,item):
        temp = Node(item)
        temp.setNext(self.head.getNext())
        self.head.setNext(temp)
        self.count = self.count + 1
```

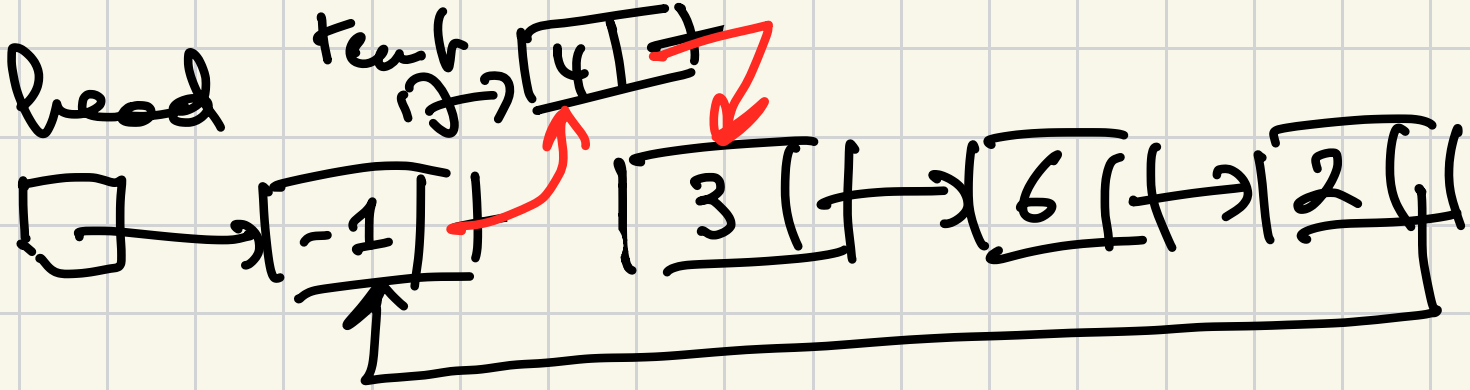


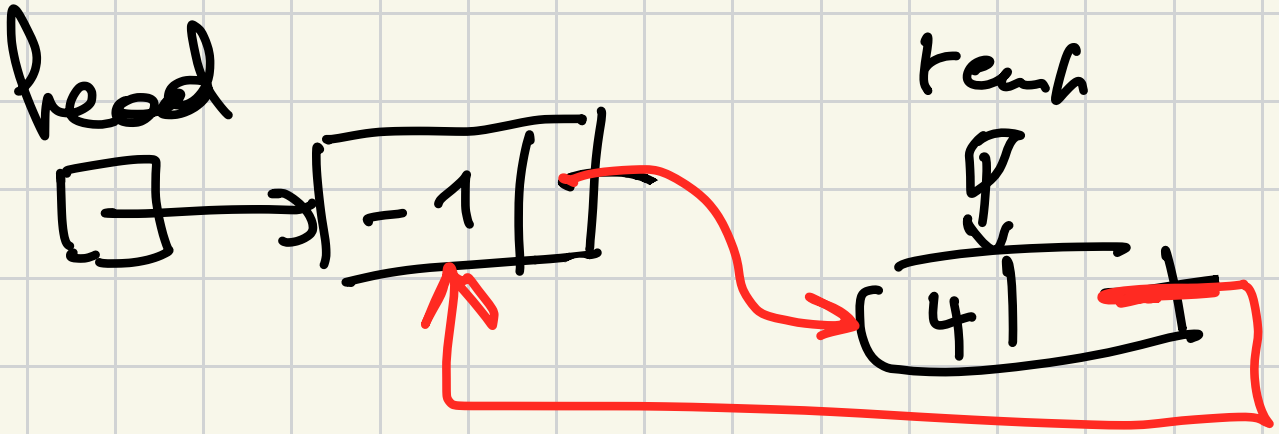
```python
def __init__(self):
    self.head = Node(-1)
    self.head.setNext(self.head)
    self.count = 0
```





```python
def add(self,item):
    temp = Node(item)
    temp.setNext(self.head.getNext())
    self.head.setNext(temp)
    self.count = self.count + 1
```

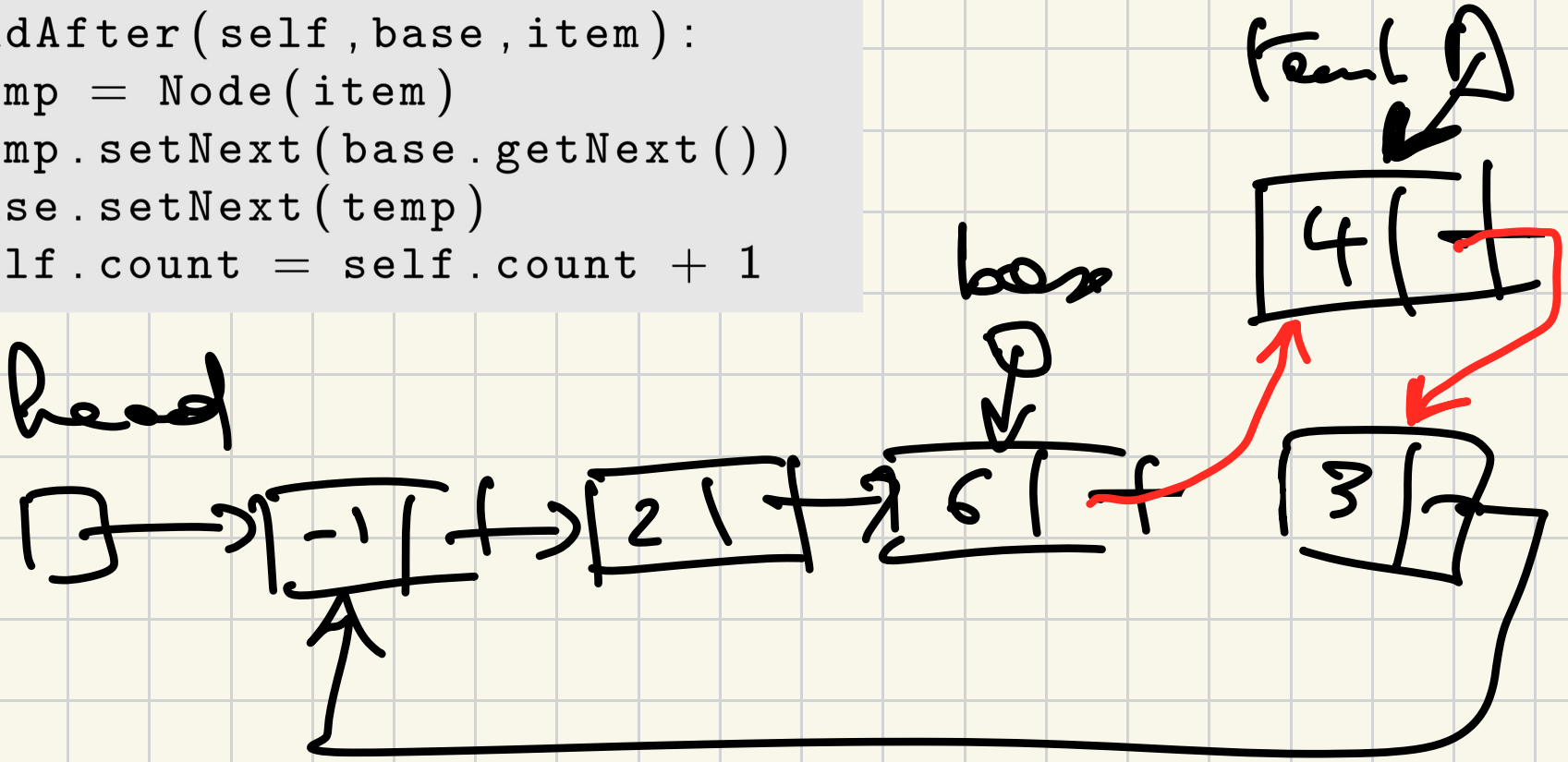




Liste circulaire

```
    def addAfter(self,base,item):
        temp = Node(item)
        temp.setNext(base.getNext())
        base.setNext(temp)
        self.count = self.count + 1

    def length(self):
        return self.count

    def search(self,item):
        current = self.head.getNext()
        found = False
        while current != self.head and not found:
            if current.getData() == item:
                found = True
            else:
                current = current.getNext()
        return found
```

```python
def addAfter(self,base,item):
    temp = Node(item)
    temp.setNext(base.getNext())
    base.setNext(temp)
    self.count = self.count + 1
```



Liste circulaire

```
    def remove(self,base):
        previous = self.head
        current = self.head.getNext()
        found = False
        while current != self.head and not found:
            if current is base:
                found = True
            else:
                previous = current
                current = current.getNext()
        if found:
                previous.setNext(base.getNext())
                self.count = self.count - 1
```

Notation postfixe

Soit une expression algébrique écrite en notation infixe (ou notation algébrique directe) :

$$(a+b)*c+d$$

Il est possible d'écrire cette expression en mettant les opérateurs après ses arguments, il s'agit de la notation postfixe (ou notation polonaise inversée) :

$$a\ b\ +\ c\ *\ d\ +$$

Cette notation ne nécessite pas l'usage de parenthèses

Evaluation d'une expression algébrique

Soit l'expression algébrique :

$$a * ((b + c) * (d + e) + f)$$

qui en postfixe peut s'écrire :

$$a \ b \ c \ + \ d \ e \ + \ * \ f \ + \ *$$

Pour évaluer cette expression (écrite en postfixe), nous pouvons la traiter en la parcourant de gauche à droite

Evaluation d'une expression algébrique

$$a\ b\ c\ +\ d\ e\ +\ *\ f\ +\ *$$

- sauvegarder en mémoire temporaire a, b puis c
- $+$: récupérer, tout en les effaçant de la mémoire temporaire, les deux *derniers* éléments sauvegardés (b et c), les sommer et sauvegarder le résultat
- sauvegarder d et e
- $+$: récupérer les deux *derniers* éléments sauvés (d et e), les sommer et sauvegarder le résultat
- $*$: récupèrer les deux *derniers* éléments sauvés ($d+e$ et $b+c$), les multiplier et sauvegarder le résultat
- sauvegarder f
- $+$: récupérer les deux *derniers* éléments sauvés (f et $(b+c)*(d+e)$), les sommer et sauvegarder le résultat
- $*$: récupèrer les deux *derniers* éléments sauvés ($(b+c)*(d+e)+f$ et a), les multiplier et sauvegarder le résultat

Evaluation d'une expression algébrique

La sauvegarde fonctionne donc suivant le principe *dernier sauvé, premier repris* (*last in, first out* en anglais)

Une telle structure de données est la *pile* (ou *stack* en anglais)

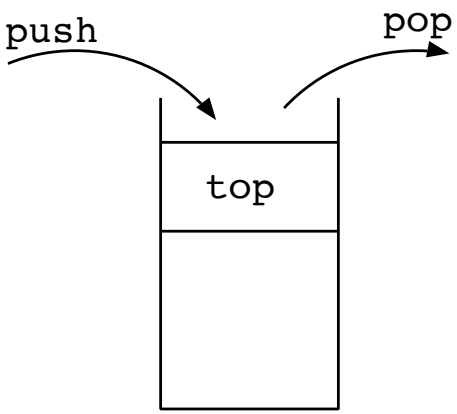
Pile

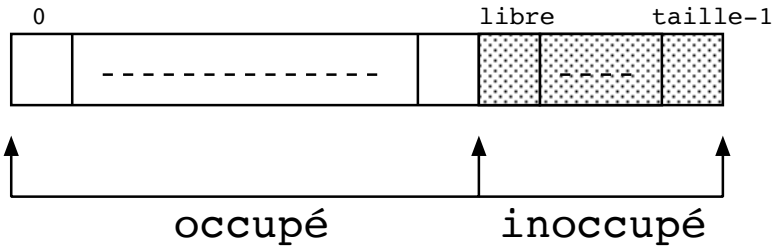
L'interface de base d'une pile consiste en les primitives suivantes :

- push : rajout d'un élément en haut de la pile
- pop : retrait d'un élément du haut de la pile
- top : indique la valeur de l'élément se trouvant « au sommet » de la pile
- isEmpty : indique si la pile est vide ou non
- size : indique le nombre d'éléments présents dans la pile

Pile au moyen d'un tableau

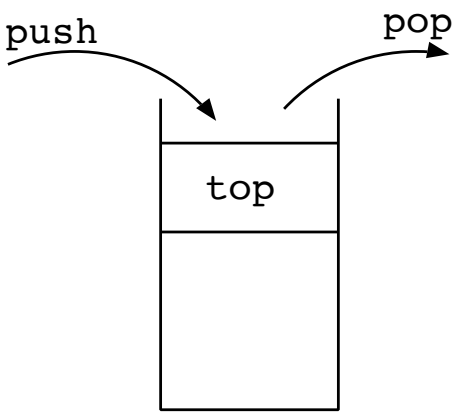
Une Pile est un type de donnée abstrait qui peut être réalisé au moyen d'un tableau

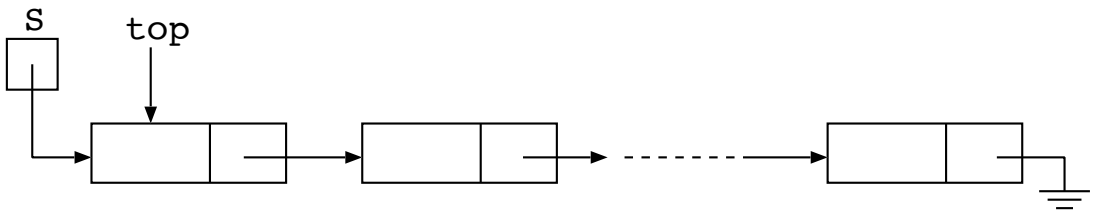




Pile au moyen de pointeurs

Une Pile est un type de donnée abstrait qui peut être réalisé au moyen de pointeurs





Pile

```
class Stack:
    def __init__(self):
        self.items = []

    def isEmpty(self):
        return self.items == []

    def push(self, item):
        self.items.append(item)

    def pop(self):
        return self.items.pop()

    def top(self):
        return self.items[len(self.items)-1]

    def size(self):
        return len(self.items)
```

Pile

```
class Stack:
    def __init__(self):
        self.items = []

    def isEmpty(self):
        return self.items == []

    def push(self, item):
        self.items.insert(0,item)

    def pop(self):
        return self.items.pop(0)

    def top(self):
        return self.items[0]

    def size(self):
        return len(self.items)
```

Autre implémentation de Pile

Utilise la classe Node

```
class Stack:
    def __init__(self):
        self.head = None
        self.n = 0

    def isEmpty(self):
        return self.head == None

    def top(self):
        return self.head.getData()

    def size(self):
        return(self.n)
```

Autre implémentation de Pile

```
    def push(self, item):
        p = Node(item)
        p.setNext(self.head)
        self.head = p
        self.n += 1

    def pop(self):
        res = self.head.getData()
        self.head = self.head.getNext()
        self.n -= 1
        return res
```

Evaluation d'une expression postfixe

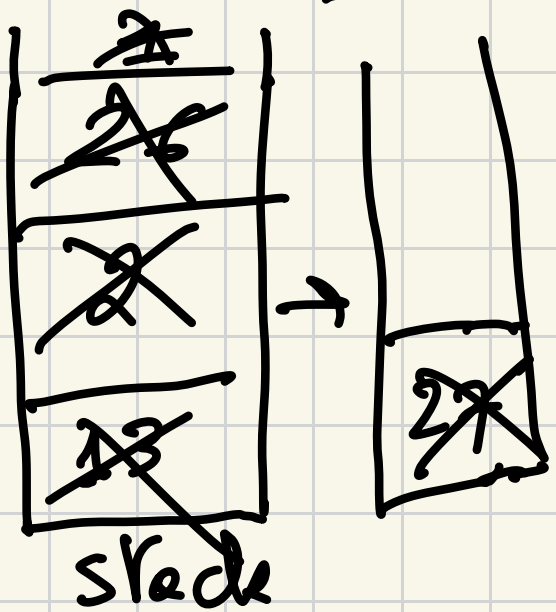
```
def doMath(op, op1, op2):
    if op == "*":
        return op1 * op2
    elif op == "/":
        return op1 / op2
    elif op == "+":
        return op1 + op2
    else:
        return op1 - op2
```

Evaluation d'une expression postfixe

```
def postfixEval(postfixExpr):
    operandStack = Stack()
    tokenList = postfixExpr.split()
    for token in tokenList:
        if token[0] in "0123456789":
            operandStack.push(int(token))
        else:
            operand2 = operandStack.pop()
            operand1 = operandStack.pop()
            result = doMath(token,operand1,operand2)
            operandStack.push(result)
    return operandStack.pop()
```

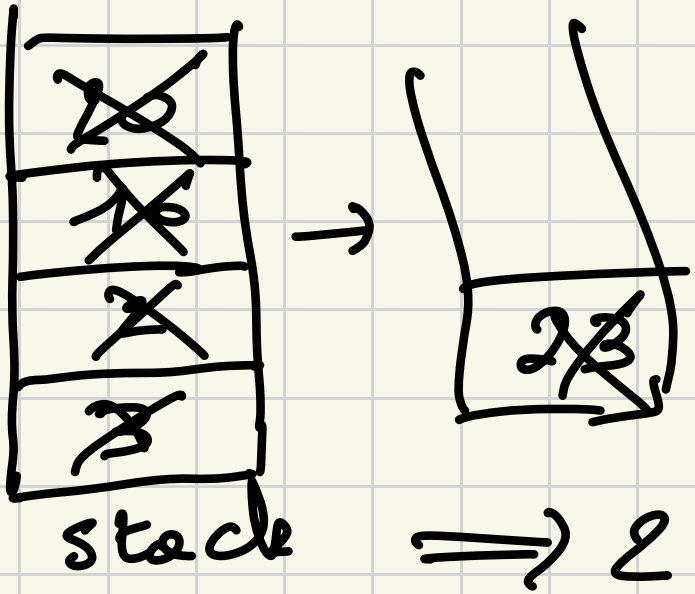
```python
def postfixEval(postfixExpr):
    operandStack = Stack()
    tokenList = postfixExpr.split()
    for token in tokenList:
        if token[0] in "0123456789":
            operandStack.push(int(token))
        else:
            operand2 = operandStack.pop()
            operand1 = operandStack.pop()
            result = doMath(token,operand1,operand2)
            operandStack.push(result)
    return operandStack.pop()
```

"13_2_X_1_+"



$\Rightarrow$ 27

"3_2_10_X_+"



$\Rightarrow$ 2

Vérification du parenthèsage

```
def parChecker(chaine):
    s = Stack()
    balanced = True
    i = 0
    while i < len(chaine) and balanced:
        symbol = chaine[i]
        if symbol == "(":
            s.push(symbol)
        elif symbol == ")":
            if s.isEmpty():
                balanced = False
            else:
                s.pop()
        i = i + 1
    if balanced and s.isEmpty():
        return True
    else:
        return False
```

Vérification du parenthèsage

```
def parChecker(chaine):
    balanced = True
    i = 0
    cpt = 0
    while i < len(chaine) and balanced:
        symbol = chaine[i]
        if symbol == "(":
            cpt = cpt + 1
        elif symbol == ")":
            if cpt == 0:
                balanced = False
            else:
                cpt = cpt - 1
        i = i + 1
    if balanced and cpt == 0:
        return True
    else:
        return False
```

Matching général

```
def parChecker(chaine):
    s = Stack()
    balanced = True
    i = 0
    while i < len(chaine) and balanced:
        symbol = chaine[i]
        if symbol in "([{":
            s.push(symbol)
        elif symbol in ")]}":
            if s.isEmpty():
                balanced = False
            else:
                top = s.pop()
                if not matches(top,symbol):
                        balanced = False
        i = i + 1
    if balanced and s.isEmpty():
        return True
    else:
        return False
```

Matching général

```
def matches(open,close):
    opens = "([{"
    closers = ")]}"
    return opens.index(open) == closers.index(close)
```

Conversion infixe vers postfixe

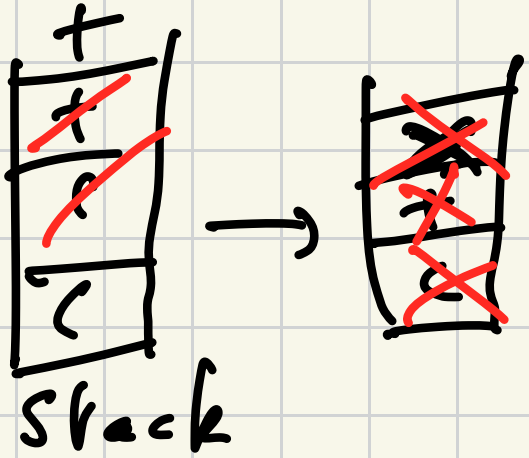
```
import string

def infixToPostfix(infixexpr):
    prec = {}
    prec["*"] = 3
    prec["/"] = 3
    prec["+"] = 2
    prec["-"] = 2
    prec["("] = 1
    s = Stack()
    result = []
```

Conversion infixe vers postfixe

```
    infixexpression = infixexpr.split()
    for token in infixexpression:
        if token in string.lowercase or token in string.↩
            digits:
            result.append(token)
        elif token == '(':
            s.push(token)
        elif token == ')':
            topToken = s.pop()
            while topToken != '(':
                result.append(topToken)
                topToken = s.pop()
        else:
            while (not s.isEmpty()) and (prec[s.top()] >= ↩
                prec[token]):
                result.append(s.pop())
            s.push(token)
    while not s.isEmpty():
        result.append(s.pop())
    return string.join(result)
```

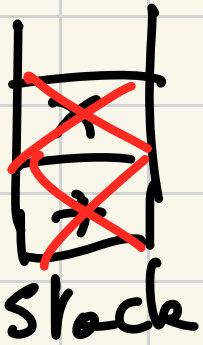
```python
    infixexpression = infixexpr.split()
    for token in infixexpression:
        if token in string.lowercase or token in string.↩
            digits:
            result.append(token)
        elif token == '(':
            s.push(token)
        elif token == ')':
            topToken = s.pop()
            while topToken != '(':
                result.append(topToken)
                topToken = s.pop()
        else:
            while (not s.isEmpty()) and (prec[s.top()] >= ↩
                prec[token]):
                result.append(s.pop())
            s.push(token)
    while not s.isEmpty():
        result.append(s.pop())
    return string.join(result)
```

"((1+2)+3×4)"



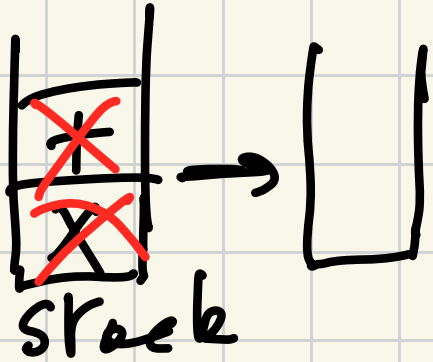
1 2 + 3 4 × +

"1+2×3"



1 2 3 × +

"1×2+3"



1 2 × 3 +

File

Un autre concept fondamental est la *file d'attente* (ou *queue* en anglais)

Une file est, comme l'est une pile, une zone de sauvegarde temporaire

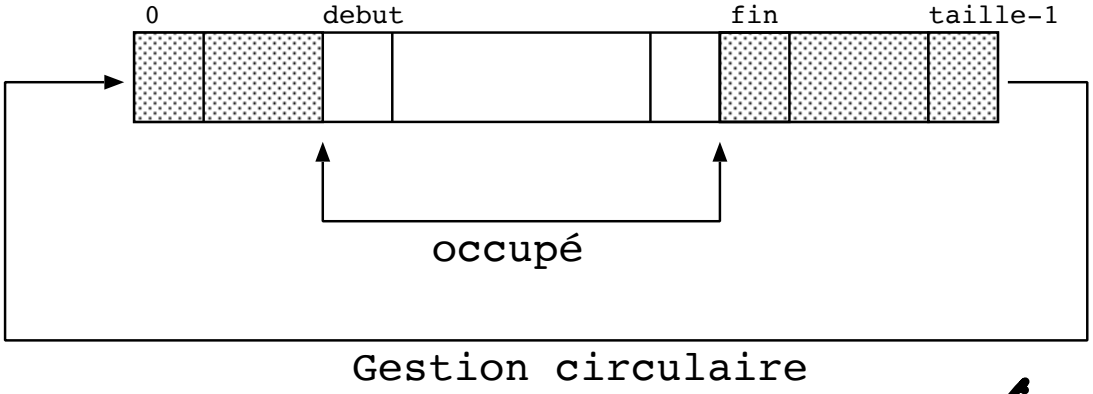
La gestion des entrées et sorties dans cette zone répond au principe *premier arrivé, premier à sortir* (*first in, first out* en anglais, ou encore *fifo* pour faire court)

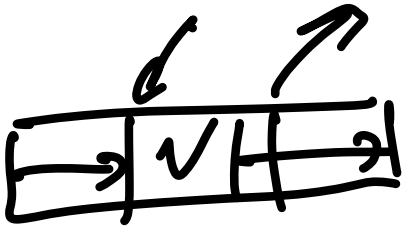
File

L'interface de base d'une file consiste en les primitives suivantes :

- insert : rajout d'un élément dans la file
- remove : retrait d'un élément de la file
- head : indique la valeur de l'élément qui sera le prochain à sortir de la file
- isEmpty : indique si la file est vide ou non
- size : indique le nombre d'éléments présents dans la file

Une File est un type de donnée abstrait qui peut être réalisé au moyen d'un tableau





File

Au moyen d'un tableau, on maintient deux indices appelés « debut » et « fin »

Le tableau est géré circulairement

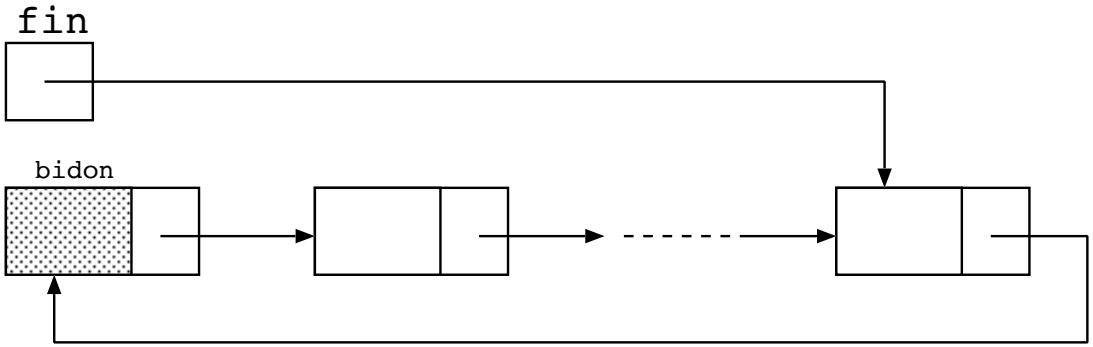
On insère un élément en « fin » (et on incrémente cet indice)

Lors d'une insertion, si on atteint l'extrémité (droite) du tableau, on continue circulairement à l'indice 0 (si cette position est libre)

On retire un élément de la file en « début » et on incrémente aussi cet indice que l'on gère aussi bien sûr circulairement

File

Une File est un type de donnée abstrait qui peut être réalisé au moyen de pointeurs



File

Au moyen d'une liste circulaire avec élément bidon, on supprime en tête de liste et on insère en fin de liste

L'élément bidon permet de s'assurer qu'il y a toujours un élément qui précède l'élément courant

On gère un pointeur « fin » qui pointe vers le dernier élément de la liste (ce pointeur nous permet aisément de retrouver la tête de la liste)

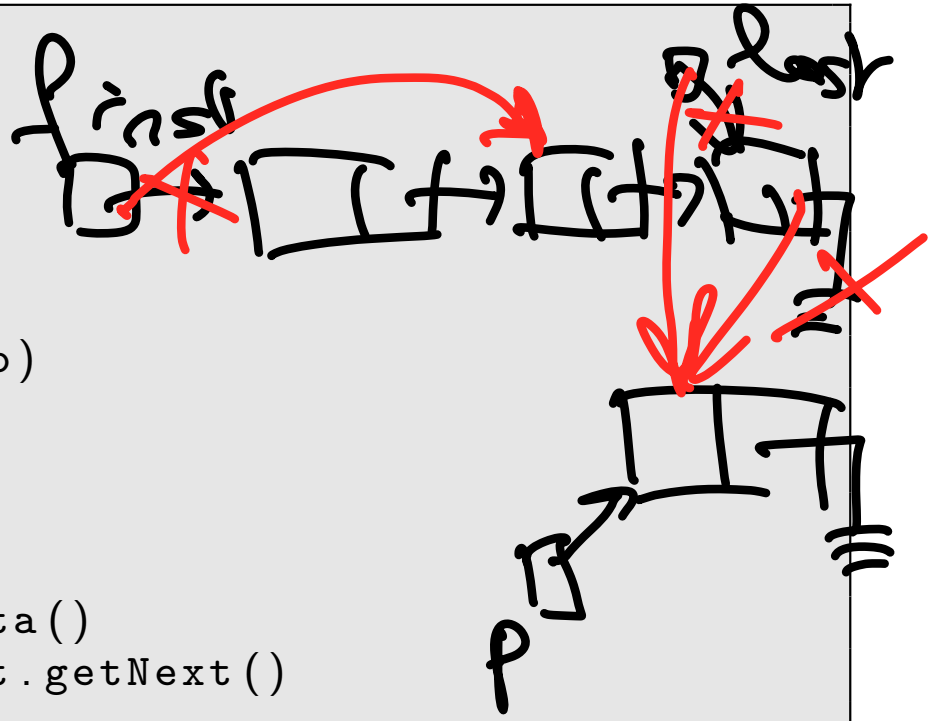
File

```
class Queue:
    def __init__(self):
        self.items = []

    def head(self):
        return self.items[len(self.items)-1]

    def isEmpty(self):
        return self.items == []

    def insert(self, item):
        self.items.insert(0,item)

    def remove(self):
        return self.items.pop()

    def size(self):
        return len(self.items)
```

Autre implémentation de File

Utilise la classe Node

```
class Queue:
    def __init__(self):
        self.first = None
        self.last = None
        self.n = 0

    def isEmpty(self):
        return self.first == None

    def size(self):
        return self.n

    def head(self):
        return self.first.getData()
```

Autre implémentation de File

```
    def insert(self, item):
        p = Node(item)
        if self.isEmpty():
            self.first = p
            self.last = p
        else:
            self.last.setNext(p)
            self.last = p
        self.n += 1

    def remove(self):
        res = self.first.getData()
        self.first = self.first.getNext()
        self.n -= 1
        return res
```